# GraMR—Graph-based Music Recommendations: Playlist Generation using a Graph Neural Network

**Chris Leinweber**
Department of Computer Science
Rice University
cl158@rice.edu

**Andrew Meaux**
Department of Computer Science
Rice University
apm7@rice.edu

## Abstract

As audio and video entertainment have moved from physical media to local files to online streaming, recommendation systems have become an integral part of the user experience. These systems focus on delivering high-quality content based on users' past activity. In this study, our team uses a graph neural network to generate novel music playlists based on publicly available Spotify data. We demonstrate that, scaled with sufficient user data, a well-trained graph neural network can consistently generate high-quality playlists based on user-selected 'seed' songs.

## 1 Introduction

Over the past decade, the entertainment industry has rapidly transitioned from physical disc-based media, such as CDs and DVDs, to that of streaming-based platforms for both audio and video content. As of the end of 2022, Statista [2022] estimated that Spotify had just over 200 million premium subscribers worldwide, a near threefold increase from just five years prior. With this rapid growth in user counts, streaming giants like Spotify and Netflix have accumulated vast amounts of user data. The ability to extract value from that data, such as through the development of recommendation systems, grows ever more important. Recommendation systems are critical to a streaming service's ability to tailor their product to consumer preferences. To this effect, our team created a music recommendation system for playlist generation using a graph neural network.

This project was first inspired by McFee and Lanckriet [2012], in which playlists were generated "based upon hypergraph random walks" as a product of fined-tuned playlist models based on "specific categories or dialects". Our goal was to expand playlist generation techniques beyond hypergraph-based random walks, hoping to create a system similar to Spotify's own. While Spotify's algorithm is not public, Tomorrow [2022] states that the company uses a combination of content-based and collaborative-based filtering, both of which can be easily represented by a graph network. For a hypergraph in which songs are nodes and playlists are hyperedges, content-based filtering would be the analysis of the node features and collaborative filtering would be the analysis of the edge features.

## 2 Data

To properly train a model to make accurate predictions, a large training dataset is required. To that end, our project used the Spotify Million Playlist Dataset, which "contains 1,000,000 playlists, including playlist titles and track titles, created by users on the Spotify platform between January 2010 and October 2017" (Spotify [2018]). A dataset of this magnitude, comprised entirely of user-generated song selections, was integral to the success of our recommendation system. Our team evaluated representing the data as a hypergraph, as McFee did in 2012. Instead, we opted to convert the hypergraph into an undirected bipartite graph representation consisting of two node types, song nodes and playlist nodes, with unweighted edges providing connectivity. As seen in Figure 1, representing

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

the dataset in this form means that song nodes and playlist nodes only have connectivity between one another, i.e., a song node will never have an edge to another song node. Part of the justification for choosing a bipartite graph representation was the ability to readily implement a graph neural network. This also simplified the mechanisms for performing random sampling for training and testing.

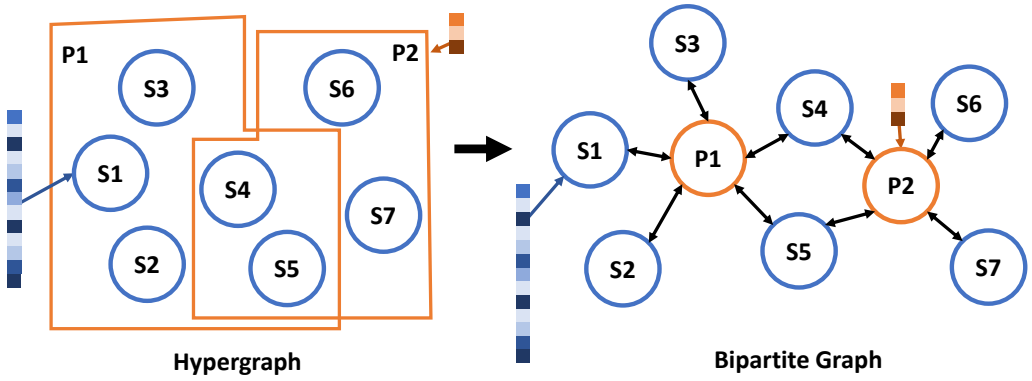
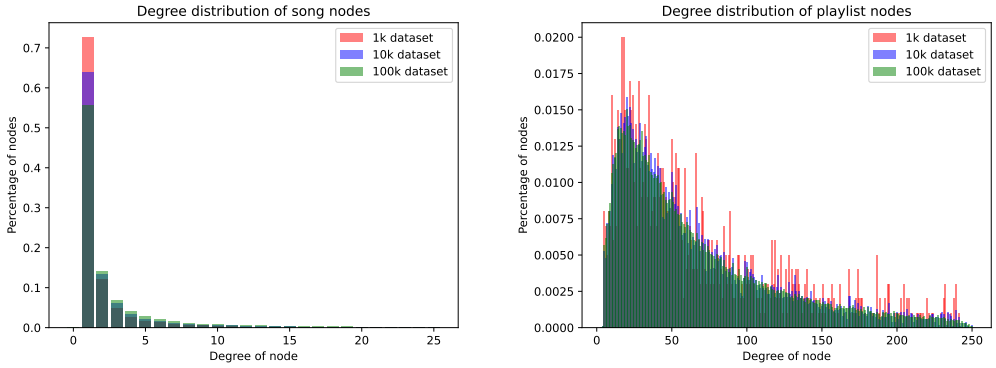



Figure 1: Hypergraph vs. bipartite graph representation.

The Spotify Million Playlist Dataset contains 1,000,000 public Spotify playlists split into 1,000 slices in JSON format. Along with playlists and their respective song lists, each slice also contains playlist features: number of tracks, artists, and followers, and song features: title, performing artist, and album. In addition to these native features, our team gathered thirteen song audio features via the Spotify API. These features included Spotify metadata such as audio features like "danceability", a song's rating of how suitable it is for dancing, and an algorithmically-derived popularity measure (Spotify [n.d.]). Due to the sheer size of the dataset, our team took a scaled approach by processing increasing numbers of slices, starting with a single slice of 1,000 playlists and building to a final set of 100,000 playlists. As seen in Figure 2, the connectivity of the graph network scaled in relation to the size of the dataset. The average playlist size remained constant at approximately 66 songs (Figure 2b), but the average degree of the song nodes increased (Figure 2) as more playlists were added. The average song node degree increased fivefold from 1.9 in the 1k dataset to 9.7 in the 100k dataset.




(a) Increasing song connectivity with larger datasets (b) Playlist size holds steady with larger datasets

Figure 2: Effects of dataset size on node degree distributions

## 3 Methods

All development for GraMR was performed using PyTorch Geometric (Fey and Lenssen [2019]). As previously mentioned, our graph network is an undirected bipartite graph comprising two node types:

songs and playlists, which have different features. The graph is represented by an HeteroData object with an incidence matrix connecting song nodes to playlist nodes, a 13D song feature matrix, and a 3D playlist feature matrix. For the 100k dataset, the data comprises 100,000 playlist nodes, 681,660 song nodes, and an astounding 6,588,495 edges. The data was divided into training, validation, and test sets containing 70%, 15%, and 15% of the data respectively. An equivalent number of negative samples was also generated for each set.

The model itself is a relatively simple multi-layer GNN (Figure 3). The model incorporates a single linear layer to transform the 13D song and 3D playlist features into a 16D hidden space, or vector embedding. This is followed by two GraphSAGE convolutional layers as described in Hamilton et al. [2018], with a single ReLU layer in between. Finally, a classifier computes the dot-product similarity of adjacent node embeddings to perform edge-level link prediction. We also defined a class method to provide access to the generated embeddings, which served as the root of our ability to generate meaningful playlists. The model was trained with random sampling of positive and negative edges to generate the node-level embeddings. Sampling was performed using a link-based extension of the neighbor sampling mechanism defined in Hamilton et al. [2018].

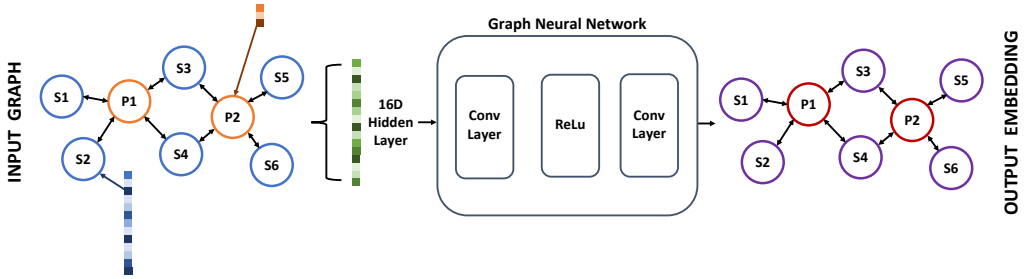



Figure 3: GraMR model pipeline.

## 4 Challenges

We knew that a sufficiently large dataset was critical to the success of our model, and the Spotify Million Playlist dataset fulfilled that need. However, the benefits of more connectivity in scaling up from 1k to 100k came with the downside of rapidly growing computational requirements. Loading the larger datasets was relatively straightforward, but extracting the additional features via the Spotify API was a lengthy task due to rate limits. After experimenting with the 100k playlist graph network, the results were promising enough to deter us from performing additional data loading and, consequently, additional feature extraction. Maintaining song and playlist information in memory to generate the graph network also approached the limits of the computer hardware. Regardless, our team maintains that additional playlist data would further improve model performance, at the cost of resource consumption.

## 5 Results

### 5.1 Metrics

To measure the validity of the trained model, the project team settled on two metrics. The first was the capability of the model to accurately classify links as either positive or negative examples. The second metric was the subjective quality of playlists generated by the model. To evaluate playlist quality, the model was used to generate novel playlists, which were then evaluated for semantic cohesion and subjective quality. Semantic cohesion in a musical context is a measure of the homogeneity of the playlist, in terms of artists, genre, and musical style. Subjective quality was evaluated by project team members as a measure of "how enjoyable the playlist would be to listen to."

### 5.2 Link Prediction

The first result we observed was that the model validity substantially improved as it was exposed to larger datasets. As discussed in Section 2 above, we generated three datasets of increasing size. The larger datasets had higher average song node degrees, leading to a more highly-connected network. As shown in Figure 4, the model AUC improved substantially with increasing dataset size. All other hyperparameters were held constant, including model depth, training duration, sampling mechanism, etc. By including two GraphSAGE convolutional layers in the model, we ensure that the audio features of song nodes are sampled and aggregated to their connected playlist nodes and then on to other song nodes. We can see how increasing the average degree of song nodes would therefore allow them to aggregate additional information from other song nodes, improving the validity of the model.

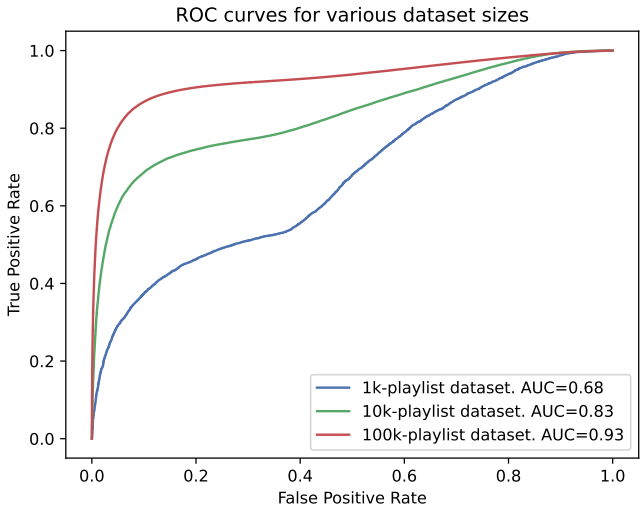



Figure 4: Model performance increases substantially with increasing dataset size

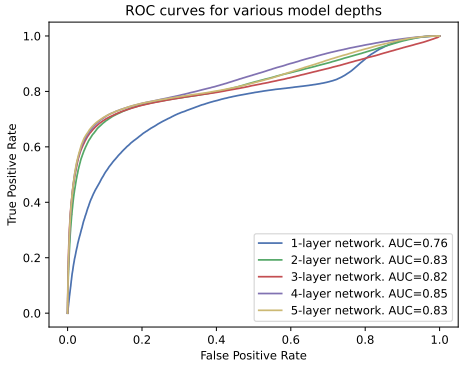
Next, we examined the effects of modifying various other model parameters and hyperparameters. The results of these experiments can be seen in Figure 5. Unless otherwise specified, all results come from a model trained on the 10k dataset (to allow for manageable computation times).

The initial model, as discussed in Section 3 above, used a 2-layer design, with 2 SAGE convolutional layers. We created alternate model architectures with differing numbers of layers, ranging from 1 to 5. The performance of the model, as shown in Figure 5a, was similar for depths between 2 and 5, with noticeable degradation when reduced to a single layer. Given the bipartite structure of the graph, a single layer only allows for messages to be passed from songs to playlists or from playlists to songs. It is intuitive that this model would not perform as well as one that allows song embeddings to be learned using information from the other songs which appear in the same playlists (song->playlist->song). Interestingly, it also seems that odd-layered models slightly underperform their equivalents with one fewer layer. This implies that there is no value gained by aggregating information from additional nodes of the opposite type (e.g. playlist->song->playlist->song).

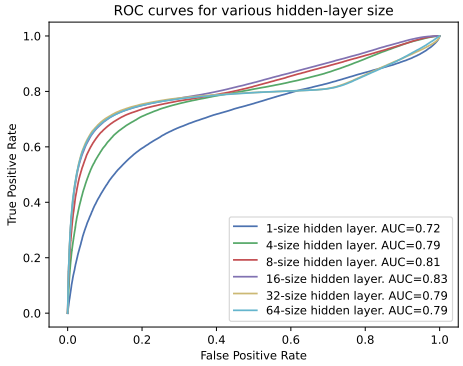
Figure 5b shows the performance of the model as the size of the vector embedding was modified. There is a clear improvement as the size is increased from 1 to 16 dimensions, and then a slight drop-off in performance with further increases to 32 or 64 dimensions. This indicates that 8-16 dimensions is a sufficient size to encode all the relevant information and properly differentiate songs. Smaller sizes, e.g. 1-4 dimensions, are not sufficient to properly distinguish songs, which can be seen when lower-dimensional models are used to generate playlists. Larger sizes may be starting to show signs of overfitting, as the model begins to have a higher false positive rate.

Figures 5c and 5d show the effects of modifying the sampling size on the 10k and 100k datasets, respectively. By default, we sampled 10 neighbors of each training/test node, and 10 neighbors of each of these neighbors. We expected that sampling fewer neighbors would degrade model performance, and more neighbors would improve it. Ultimately, the number of neighbors appeared to have a very

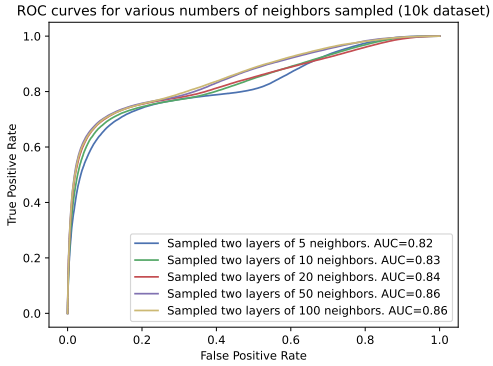
small effect on the performance of the model, within a reasonable range. On the 10k dataset, increases in the number of neighbors did show some improvement, where it appears that more neighbors may help mitigate the loss of information from using a smaller dataset. On the 100k dataset, increasing from 5 to 20 neighbors showed only minimal effects on model performance.



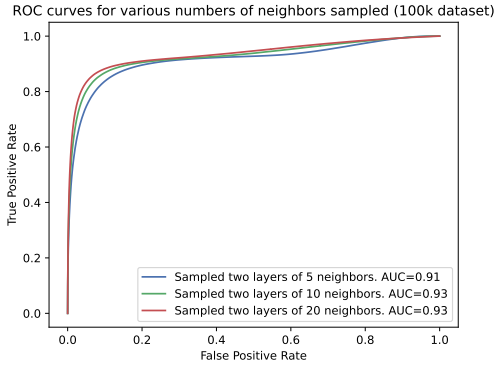

(a) Model performance vs. model depth




(b) Model performance vs. hidden layer size




(c) Model performance vs. number of neighbors sampled (10k dataset)




(d) Model performance vs. number of neighbors sampled (100k dataset)

Figure 5: Model performance with various hyperparameter adjustments

### 5.3 Playlist Generation

In addition to quantitative metrics, we wanted to evaluate how well various model iterations could generate novel playlists. To this end, we developed 4 distinct methods for playlist generation. [1] The first method, which we refer to as "Most Similar Embedding", takes a seed song and identifies the songs with the most similar model-generated embeddings (calculated by dot product). Table 1 shows the first 10 songs in a playlist generated using this method from the seed song "Sweet Child O' Mine" by Guns N' Roses. All 10 songs are semantically consistent, comprised entirely of classic rock songs (although interestingly all the added songs are from the 1970s whereas Sweet Child O' Mine was recorded in 1987).

The next playlist generation method still relies on the similarity of song embeddings, but adds some randomness into the calculation using the playlist data. Starting from a seed song, we randomly select a playlist in which the song appears. Stepping into this playlist, we then select the song in the playlist with the most similar embedding to the seed song. This can be envisioned as stepping randomly away from the seed song to a playlist node in the embedding space, then stepping as far back towards the seed song as possible given the other songs in the playlist. Table 2 shows the first 10 songs in a playlist generated using this method from the same seed song as above. Additionally,

[1] All playlists showcased here were developed using a 2-layer model with 16-dimensional vector embeddings, trained on the 100k dataset, unless otherwise noted.

Table 1: Playlist generated using "Most Similar Embedding" method

| Song | Artist |
|---|---|
| Sweet Child O' Mine | Guns N' Roses |
| Rock and Roll - Remaster | Led Zeppelin |
| Fairies Wear Boots - Live | Black Sabbath |
| Immigrant Song - Remaster | Led Zeppelin |
| Sweet Home Alabama | Various Artists |
| Free Bird | Lynyrd Skynyrd |
| Smokin' | Boston |
| Surrender | Cheap Trick |
| Hold the Line | TOTO |
| Smoke On The Water - Remastered 2012 | Deep Purple |

the "linking playlist" is shown for each song after the seed. Once again, the 10 songs are semantically consistent, and the linking playlists show that these songs frequently appear in playlists dedicated to classic rock. In one author's opinion, this playlist is even better than the first.

Table 2: Playlist generated using "Random Playlist-Similar Embedding" method

| Song | Artist | Linking Playlist |
|---|---|---|
| Sweet Child O' Mine | Guns N' Roses | N/A (Seed Song) |
| More Than a Feeling | Boston | Classic Rock |
| Have You Ever Seen The Rain | Creedence Clearwater Revival | Oldies |
| Take It Easy - 2013 Remaster | Eagles | Dad's songs |
| Listen to the Music | The Doobie Brothers | classics : ) |
| Rock'n Me | Steve Miller Band | Classic |
| Take The Money And Run | Steve Miller Band | Roots |
| The Boys Are Back In Town | Thin Lizzy | Booze Cruise |
| Rock and Roll - Remaster | Led Zeppelin | Classics |
| Closer To The Heart | Rush | Classic Rock |

Adding additional randomness to the calculation, we next attempted an implementation of a more standard Weighted Random Walk. In this iteration, we still step randomly from the seed song to a connected playlist. We also calculate the similarities of all songs in the playlist to the seed song. Instead of selecting the most similar song, we use the similarities as weights to perform the random choice of songs. Imagined in the embedding space, this would appear as an attractive force pulling our selections back towards the seed song without overwhelming the force of randomness. Table 3 shows the first 10 songs in a playlist generated using this method from the same seed song as above. We can see that the songs gradually transition further away from the seed than in either of the previous playlists. We bounce back into the 1960s with "Spirit in the Sky" (amusingly contained in a playlist called "70s") and "Hey Jude", diverge into less-well-known rock and rap, then return to 60s pop rock with "Up On The Roof".

Table 3: Playlist generated using "Weighted Random Walk" method

| Song | Artist | Linking Playlist |
|---|---|---|
| Sweet Child O' Mine | Guns N' Roses | N/A (Seed Song) |
| No More Tears | Ozzy Osbourne | Rock |
| Baba O'Riley | The Who | Yup! |
| Spirit In The Sky | Norman Greenbaum | workout tunes |
| Bang a Gong (Get It On) - Re-Recorded / Remastered | Various Artists | 70s |
| Highway to Hell | AC/DC | 70s |
| Hey Jude | The Beatles | Jams |
| Smashed Blocked(studio) | John's Children | playlist |
| Gurl | Father | playlist |
| Up On The Roof | Cryan' Shames | playlist |

Our final algorithm is a classical Random Walk. From a given seed song, we jump randomly to a connected playlist. From this playlist, a random connected song is selected. This process repeats until the playlist reaches the desired size. Table 4 shows the first 10 songs in a playlist generated using this method from the same seed song as above. The added randomness is immediately observable, as this playlist diverges much farther away from the seed song, ending in Tejano and Regional Mexican music within the first 10 songs.

Table 4: Playlist generated using "Random Walk" method

| Song | Artist | Linking Playlist |
|---|---|---|
| Sweet Child O' Mine | Guns N' Roses | N/A (Seed Song) |
| Dream On | Aerosmith | Old School |
| Crossroads - Live At Winterland, SF / 1968 | Cream | rock n' roll |
| Dead Leaves and the Dirty Ground | The White Stripes | yea |
| It Don't Move Me | Peter Bjorn and John | Spring 2013 |
| Blue Shift | Lemaitre | yah |
| Skinny Love | Bon Iver | Anything |
| Te Odio Y Te Amo | Duelo | <3 |
| Trate De Olvidarte con actuacion especial ... | Siggno | norteñas |
| Prometí | Intocable | norteñas |

Lastly, we generated a playlist using the "Most Similar Embedding" method, but with a model that had only a one-dimensional vector embedding. The first 10 songs in this playlist are shown in Table 5. The playlist shows much wider variability than the Most Similar 16D Embedding playlist in Table 1, indicating that the single dimension is not sufficient for the model to adequately differentiate between muscially different songs.

Table 5: Playlist generated using "Most Similar Embedding" method with 1D vector embeddings

| Song | Artist |
|---|---|
| Sweet Child O' Mine | Guns N' Roses |
| Oildale (Leave Me Alone) | Korn |
| Bombs | Dawn Richard |
| The Sounder | Gorillaz |
| Break The Silence | Thousand Foot Krutch |
| Woman's Work | Tracy Chapman |
| Technologic - Liquid Twins Remix | Daft Punk |
| Secrets (feat. the Weeknd) | Jr. Hi |
| Tremors | Angels & Airwaves |
| Beautiful Addiction | Nate Feuerstein |

## 6 Discussion & Future Work

GraMR is a GNN-based music recommendation system trained on publicly available user-generated playlists. When trained on a sufficiently large dataset (at least 100,000 playlists) and supplied an input seed song, GraMR can generate high-quality playlists semantically coherent with the seed song, i.e., consistent with the seed song's genre, artist, era, ambiance, etc. Additionally, various generation methods can be used to control the level of variety in the resultant playlist. There are multiple avenues for future expansion of the current work, including text embedding of the non-numerical features of the songs and playlists, modeling the graph network as a true hypergraph, including artist or album information as additional edges/hyperedges, and developing playlist generation methods based on multiple seed songs.

Although we limited our research to one-tenth of the entire Spotify Million Playlist Dataset, our team expects that future iterations with additional data would only increase GraMR's overall performance. In a world built on traditional, objective data analysis, the use of machine learning to venture into subjective topics is intriguing. Although such subjectivity can be hard to measure, the beauty of music is that expression and enjoyment are in the ear of the beholder, and GraMR is only one example of an increasingly exciting future in machine learning's exploration of the arts. While Spotify already has

built-in playlist recommendation and music discovery systems, providing listeners with additional tools to discover new music can only be seen as a positive.

## References


Matthias Fey and Jan Eric Lenssen. Fast Graph Representation Learning with PyTorch Geometric, May 2019. URL https://github.com/pyg-team/pytorch_geometric.

William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs, 2018.

Brian McFee and Gilles Lanckriet. Hypergraph models of playlist dialects. In *International Society for Music Information Retrieval Conference (ISMIR 2012)*, 2012. URL https://brianmcfee.net/papers/ismir2012_dialect.pdf. [Accessed February 9, 2023].

Spotify. Spotify million playlist dataset challenge. AIcrowd, 2018. URL https://www.aicrowd.com/challenges/spotify-million-playlist-dataset-challenge. [Accessed February 10, 2023].

Spotify. Audiofeatures interface in the spotify javascript sdk documentation, n.d. URL https://spotify.js.org/interfaces/AudioFeatures.html#speechiness. [Accessed April 22, 2023].

Statista. Spotify: number of premium subscribers worldwide 2022, 2022. URL https://www.statista.com/statistics/244995/number-of-paying-spotify-subscribers/. [Accessed April 7, 2023].

Music Tomorrow. How spotify recommendation system works - a complete guide 2022. Music Tomorrow blog, 2022. URL https://www.music-tomorrow.com/blog/how-spotify-recommendation-system-works-a-complete-guide-2022/. [Accessed April 18, 2023].